MMDU MULLANA OFFICIAL SYLLABUS • BCSE-008 • UNIT 3

Unit 3: Arrays, Strings, Functions & Modular Programming

Subject: Computational & Problem Solving Using C (BCSE-008) • **Exam Weightage:** 24 - 28 Marks • **Study Time:** 45 mins

1D & 2D arrays, mathematical derivation of row-major order addressing, Linear Search, Binary Search, Bubble Sort step trace, string library functions, 12-point storage classes matrix, and call by value vs reference.

1. 1D & 2D Arrays: Memory Mapping & Row-Major Formula Derivation

1. Array Memory Architecture

An array is a homogeneous collection of elements stored sequentially in contiguous memory locations.

- **1D Array Physical Address Formula:**

$$extAddress(A[i]) = \text{Base Address} + (i \times \text{sizeof}(T))$$

---

2. Mathematical Derivation: 2D Array Row-Major Order

In C, multi-dimensional arrays are mapped into linear RAM in **Row-Major Order** (row 0 stored first, followed by row 1, etc.).

For an array declared as `T A[R][C]` with base address `Base` and element size `S`:

- To locate element `A[i][j]`, the program must skip `i` full preceding rows.
- Each row contains exactly `C` columns (elements).
- Within row `i`, the program must skip `j` preceding column elements.

$$extTotalElementsSkipped = (i \times C) + j$$

$$\mathbf{\text{Address}(A[i][j]) = \text{Base Address} + \left[(i \times C) + j\right] \times \text{sizeof}(T)}$$

✅ **FUNDAMENTAL LAW / RULE:**
Numerical Examination Example:

> Given `int A[4][5]` starting at `Base = 2000` with `sizeof(int) = 4` bytes:

> Find the physical address of element `A[2][3]` :

> - `i = 2, C = 5, j = 3, S = 4`

> - $extOffset = (2 \times 5 + 3) \times 4 = 13 \times 4 = 52$ bytes

> - $extAddress(A[2][3]) = 2000 + 52 = mathbf2052$

2. Searching & Sorting Algorithms: Linear, Binary & Bubble Sort

Searching & Sorting Master Comparative Matrix

Algorithm	Preconditions	Best Time	Worst Time	Space	Core Operational Principle
Linear Search	None (Unordered)	$O(1)$	$O(N)$	$O(1)$	Sequentially compares search key against indices 0 to $N-1$.
Binary Search	**Array must be sorted!**	$O(1)$	$O(logN)$	$O(1)$	Computes `mid = low + (high-low)/2` ; halves search space each step.
Bubble Sort	None	$O(N)$ (optimized)	$O(N^2)$	$O(1)$	Repeatedly compares adjacent elements and swaps them, sinking maximum to the end.

Bubble Sort Step-by-Step Pass Trace

Tracing `[64, 34, 25, 12, 22, 11, 90]` :

- **Initial:** `[64, 34, 25, 12, 22, 11, 90]`
- **Pass 1:** Sinks **90** to index 6 ➔ `[34, 25, 12, 22, 11, 64, 90]`
- **Pass 2:** Sinks **64** to index 5 ➔ `[25, 12, 22, 11, 34, 64, 90]`

- **Pass 3:** Sinks **34** to index 4 ➔ `[12, 22, 11, 25, 34, 64, 90]`
- **Pass 4:** Sinks **25** to index 3 ➔ `[12, 11, 22, 25, 34, 64, 90]`
- **Pass 5:** Sinks **22** to index 2 ➔ `[11, 12, 22, 25, 34, 64, 90]`
- **Pass 6:** Zero swaps detected ➔ **Array completely sorted in $O(N)$ optimized time!**

3. Exhaustive 12-Point Master Storage Classes Matrix

Complete 12-Point Storage Classes Matrix

Storage Class	Keyword	Storage Location	Initial Default	Scope / Visibility	Lifetime	Linkage	Exam Key Behavior
Automatic	`auto`	Stack Memory	Garbage value	Local to block	Block execution	None	Default for all local variables. Destroyed upon function return.
Register	`register`	CPU Microprocessor Register	Garbage value	Local to block	Block execution	None	Fast access hint for loop counters. Address-of `&var` is ILLEGAL!
Static	`static`	Data Segment (BSS / Data)	Zero (`0` / `'0'`)	Local to block or file	Entire program run	Internal	Retains value across function invocations. Initialized once at compile time.

| **External** | `extern` | Data Segment | Zero (`0` / `'0'`) | Global across all files | Entire program run | External | References a global variable defined in another file. Allocates no new memory. |

4. Parameter Passing: Call by Value vs Reference & Recursion

1. Call by Value vs. Call by Reference

Feature	Call by Value	Call by Reference
Passed Entity	Copy of actual argument value is passed.	Memory address of variable (`&x`) is passed via pointers.
Caller Modification	Mutations to formal parameters do NOT affect caller.	Dereferencing (`p`) *directly mutates caller variable.*
Memory Cost	Duplicates memory for large structs.	Zero copying overhead; passes a single 4/8-byte pointer.
C Syntax	`void swap(int a, int b)`	`void swap(int a, int *b)`

---

2. Recursion & Call Stack Frame Execution

Every recursive invocation allocates a new **Stack Frame** on the process stack. Execution must have a well-defined **Base Condition**; omitting it causes **Stack Overflow**:

```
Calling fact(3):
[ Stack Frame 3: fact(3) waiting on 3 * fact(2) ] (Pushed)
[ Stack Frame 2: fact(2) waiting on 2 * fact(1) ] (Pushed)
[ Stack Frame 1: fact(1) hits base condition n <= 1 ] (Returns 1)
---------------- Unwinding Phase ----------------
Frame 2 receives 1 ➔ computes 2 * 1 = 2 (Popped)
Frame 3 receives 2 ➔ computes 3 * 2 = 6 (Popped ➔ Final Result = 6)
```

5. University Examination Solved Questions (Unit 3)

University Solved Examination Problems (MMDU Pattern)

Question 7 (1 Mark): 2D Row-Major Address Calculation

Question: In a row-major 2D array `int A[3][4]` starting at `Base = 1000` with `sizeof(int) = 4`, find the linear address of `A[1][2]`.

Step-by-Step Solution:

> $$extOffset = (i \times C + j) \times \text{Size} = (1 \times 4 + 2) \times 4 = 6 \times 4 = 24 \text{ bytes}$$

> $$extPhysicalAddress = 1000 + 24 = mathbf{1024}$$

Question 15 (2 Marks): Swap Implementation via Call by Reference

Question: Implement the classic integer swap function using pointers and explain why Call by Value fails.

Step-by-Step Solution:

```
// Call by Reference Swap:
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
} // In main: swap(&x, &y);
```

Call by Value passes copies of `x` and `y`; mutating the copies leaves the caller's variables unchanged. Call by Reference passes memory addresses `&x` and `&y`, mutating actual values in the caller's stack frame.

Question 19 (4 Marks): Binary Search Implementation & Complexity

Question: Present the Binary Search algorithm and prove its $O(logN)$ time complexity.

Step-by-Step Solution:

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Prevents arithmetic overflow
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // Not found
}
```

Proof: Each comparison step halves the remaining array interval: $N \rightarrow N/2 \rightarrow N/4... \rightarrow N/2^k = 1$. Solving $2^k = N$ gives $k = log_2 N$. Maximum comparisons are $O(logN)$.